

# SBNZ - prijedlog projekta

SV2-2022 Milan Lazarević

---

**Tema:** UAV GeoSurvey Decision Support System

## 1. Motivacija

Upotreba dronova u geodeziji, fotogrametriji i mapiranju terena postaje sve zastupljenija. Prije je isti posao obavljan avionima što je bilo rijetko i skupo. Uz pomoć dronova ostvarujemo precizne rezultate uz dosta manje materijalnih sredstava.

Tokom leta, dron generiše veliki broj telemetrijskih podataka (**baterija, signal, visina, brzina**), koje operator mora pratiti i interpretirati u realnom vremenu (nastaviti misiju, vratiti se ili sleteti). Takođe, pored telemetrije dron prima signal sa satelita i sistem na osnovu toga treba da odlučuje ( loš signal -> obustavi let)

Međutim, ljudska procjena može biti spora i podložna greškama, naročito u situacijama kada se više rizičnih faktora javlja istovremeno. Što može dovesti do ljudskih povreda ili velike materijalne štete. Zbog toga postoji potreba za sistemom koji može automatski analizirati podatke i donositi odluke na osnovu ekspertskog znanja.

Ovaj projekat ima za cilj razvoj sistema baziranog na znanju koji omogućava detekciju rizičnih stanja drona i predlaganje adekvatnih akcija u realnom vremenu.

---

## 2. Pregled problema

Sistem je namjenjen za UAV geodetsko snimanje i integriše telemetriju drona sa GNSS parametrima kako bi procenio kvalitet leta i doneo odgovarajuću operativnu odluku. Fokus je na fotogrametrijskim misijama gde kvalitet podataka i bezbednost imaju ključnu ulogu

Postojeća rješenja u praksi uglavnom:

- **DJI FlightHub 2** ([link](#))- flight management system - napredna platforma za vođenje autonomnog leta za različite misije. Koristi se za inteligentno planiranje leta, ima mogućnost integracije third party biblioteka. Podržava automatski RTH (Return to

Home) na osnovu nivoa baterije i gubitka signala. Nema kombinovanu analizu faktora niti objašnjavanje odluka.

- **ArduPilot / Mission Planner** Najpopularniji open-source autopilot. Podržava failsafe akcije (RTH, Land) na osnovu pojedinačnih pragova – baterija, RC signal, GNSS. Nema višefaktorske analize, nema objašnjenja.

DJI sistem je je closed-source i iziskuje rad isključivo sa DJI markom letjelica, dok je ardupilot opensource, može da se koristi na različitim letjelicama, ali su mogućnosti jako male.

Sistem će imati uvid u telemetrijske podatke u realnom vremenu i time će moći da reaguje i na kompleksnije kombinacije podataka. Prednost sistema je u tome što modeluje način razmišljanja eksperta i omogućava fleksibilno proširenje baze znanja.

---

## 3. Metodologija rada

### 3.1 Ulaz u sistem (Input)

Ulaz u sistem predstavlja tok telemetrijskih podataka (event stream):

- nivo baterije (%)
- jačina signala
- visina leta
- brzina kretanja
- vremenski pečat (timestamp)
- kvalitet GNSS signala (HDOP, broj satelita, tip fixa),

Podaci se modeluju kao događaji koji se obrađuju u realnom vremenu.

---

### 3.2 Izlaz iz sistema (Output)

Sistem generiše:

- stanje leta:
  - SAFE
  - WARNING
  - CRITICAL
- preporučene akcije:
  - RETURN\_HOME
  - LAND
  - CONTINUE

- SLOW\_DOWN
  - objašnjenje:
    - lista aktiviranih pravila koja su dovela do odluke
- 

### 3.3 Baza znanja

Baza znanja se sastoji od pravila organizovanih u više nivoa:

#### Nivo 1 – osnovne činjenice

- detekcija pojedinačnih stanja:
  - nizak nivo baterije
  - slab signal
  - prevelika brzina
  - kvalitet GNSS signala (HDOP, broj satelita, tip fixa),

**when** BatteryReading(level < 10)

**then** insert(new FlightFact("BATTERY\_CRITICAL")); insert(new FlightWarning("Baterija ispod 10%"));

**when** BatteryReading(level >= 10, level < 20) not FlightFact(value == "BATTERY\_CRITICAL")

**then** insert(new FlightFact("BATTERY\_LOW")); insert(new FlightWarning("Baterija ispod 20%"));

**when** SignalReading(strength == 0)

**then** insert(new FlightFact("SIGNAL\_LOST")); insert(new FlightWarning("Signal potpuno izgubljen"))

**when** SignalReading(strength > 0, strength < 25) not FlightFact(value == "SIGNAL\_LOST")

**then** insert(new FlightFact("SIGNAL\_WEAK")); insert(new FlightWarning("Slab signal"));

**when** SpeedReading(speed > 15)

**then** insert(new FlightFact("OVERSPEED")); insert(new FlightWarning("Prekoračena maksimalna brzina"));

**when** GnssReading(hdop > 5.0)

**then** insert(new FlightFact("GNSS\_POOR\_HDOP")); insert(new FlightWarning("HDOP > 5, pozicija nepouzdana"));

**when** GnssReading(satellites < 4)

**then** insert(new FlightFact("GNSS\_FEW\_SATS")); insert(new FlightWarning("Premalo satelita (< 4)"));

**when** GnssReading(fixType == FixType.NONE)

**then** insert(new FlightFact("GNSS\_NO\_FIX")); insert(new FlightWarning("Nema GNSS fixa"));

## Nivo 2 – izvedena stanja

- kombinacija osnovnih faktora:
  - rizično stanje leta
  - nestabilan let
  - los signal

**when** FlightFact(value == "BATTERY\_LOW") FlightFact(value == "SIGNAL\_WEAK")

**then** insert(new FlightFact("FLIGHT\_RISKY"));

**when** FlightFact(value == "OVERSPEED") (FlightFact(value == "GNSS\_POOR\_HDOP") or FlightFact(value == "GNSS\_FEW\_SATS"))

**then** insert(new FlightFact("FLIGHT\_UNSTABLE")); insert(new FlightWarning("Nestabilan let: brzina + slab GNSS"));

**when** FlightFact(value == "GNSS\_POOR\_HDOP") FlightFact(value == "GNSS\_FEW\_SATS")

**then** insert(new FlightFact("GNSS\_UNRELIABLE"));

**when** \$e1 : BatteryReading(\$lvl1 : level, \$t1 : timestamp)

\$e2 : BatteryReading( level < (\$lvl1 - 12),this after[0s, 30s] \$e1 )

**then** insert(new FlightFact("BATTERY\_DRAIN\_RAPID"));

insert(new FlightWarning( "Nagli pad baterije: " + \$lvl1 + "% → " + \$e2.getLevel() + "% za 30s"));

**when** accumulate(FlightWarning() over window:time(120s),\$count : count(1)) eval(\$count >= 4)

**then** insert(new FlightFact("HIGH\_WARNING\_FREQUENCY"));

## Nivo 3 – odluke

- donošenje konačnih odluka:
  - kritično stanje
  - preporučene akcije

**when** FlightFact(value == "SIGNAL\_LOST") not FlightDecision(action == "LAND")

```
then insert(new FlightDecision(FlightState.CRITICAL,Action.LAND, "Signal potpuno
izgubljen – hitno sletanje"));

when FlightFact(value == "BATTERY_CRITICAL") FlightFact(value == "SIGNAL_WEAK")

then insert(new FlightDecision(FlightState.CRITICAL, Action.RETURN_HOME,"Kritična
baterija uz slab signal – Return to Home" ));

when FlightFact(value == "GNSS_NO_FIX") not FlightDecision(action == "LAND")

then insert(new FlightDecision(FlightState.CRITICAL,Action.LAND,"Nema GNSS fixa – let
nije siguran"));

when FlightFact(value == "BATTERY_DRAIN_RAPID") not FlightDecision()

then insert(new FlightDecision(FlightState.CRITICAL,Action.RETURN_HOME, "Detektovan
nagli pad baterije – Return to Home"));

when FlightFact(value == "FLIGHT_RISKY") not FlightFact(value ==
"BATTERY_CRITICAL")not FlightDecision(state == FlightState.CRITICAL)

then insert(new FlightDecision( FlightState.WARNING, Action.RETURN_HOME, "Rizično
stanje: baterija + signal" ));

when FlightFact(value == "OVERSPEED") not FlightDecision(state ==
FlightState.CRITICAL)

then insert(new FlightDecision(FlightState.WARNING, Action.SLOW_DOWN, "Prekoračena
brzina – usporiti" ));

when FlightFact(value == "FLIGHT_UNSTABLE") not FlightDecision(state ==
FlightState.CRITICAL)

then insert(new FlightDecision(FlightState.WARNING,Action.SLOW_DOWN, "Nestabilan let
– smanjiti brzinu"));

when FlightFact(value == "HIGH_WARNING_FREQUENCY") not FlightDecision(state ==
FlightState.CRITICAL)

then insert(new FlightDecision(FlightState.WARNING, Action.RETURN_HOME, "Visoka
učestalost upozorenja u zadnja 2 minuta"));

when not FlightFact(value contains "CRITICAL" || value == "SIGNAL_LOST" || value ==
"FLIGHT_RISKY" || value == "GNSS_UNRELIABLE") not FlightDecision()

then insert(new FlightDecision( FlightState.SAFE, Action.CONTINUE, "Svi parametri u
granicama – nastaviti misiju" ));
```

---

### 3.4 CEP (Complex Event Processing)

Sistem koristi CEP za obradu događaja kroz vrijeme:

- detekcija naglog pada baterije u određenom vremenskom intervalu
- detekcija učestalih warning događaja u kratkom vremenu
- analiza promjena signala kroz vrijeme

Koriste se vremenski prozori (`window:time`) i obrasci događaja. Konkretno koristili bi Sliding window koji bi bio postavljen da posmatra podatke u zadnjih  $n$  sekundi. Ovaj pristup bi se koristio prilikom agregacije brzine pražnjenja baterije. Ako baterija naglo opada to ćemo dodati kao novu činjenicu u bazu znanja.

---

### 3.5 Accumulate funkcija

Accumulate funkcija se koristi za agregaciju podataka:

- broj warning događaja u određenom vremenskom intervalu
- detekcija visokog rizika na osnovu učestalosti događaja

```
when accumulate(FlightWarning() over window:time(2m), $count : count(1))
```

```
    eval($count >= 3)
```

```
then insert(new HighFrequencyWarning($count));
```

---

### 3.6 Drools Rule Templates

Sistem koristi Drools Rule Template mehanizam za dinamičko generisanje pravila iz eksternih podataka. Umjesto da se svako pravilo piše ručno u DRL fajlu, template definiše strukturu pravila jednom, a konkretne vrijednosti pragova (thresholds) se učitavaju iz eksterne tabele (Excel, CSV, baza podataka) u runtime-u.

U sistemu postoji veliki broj strukturno identičnih pravila koja se razlikuju samo po vrijednostima pragova. Na primjer, pravila za detekciju stanja baterije imaju isti oblik – mijenja se samo numerički prag i naziv činjenice:

***kada je baterija < 10% → BATTERY\_CRITICAL***

***kada je baterija < 20% → BATTERY\_LOW***

***kada je baterija < 30% → BATTERY\_WARNING***

Bez template-a, svaki novi prag zahtijeva izmjenu DRL fajla i rekompajliranje. Sa template-om, operater može da promijeni pragove u Excel tabeli bez dodirivanja koda.

Primjer podataka koji bi se ucitavali iz (iz Excel/CSV tabele):

| parameter | operator | threshod | factName         | warningMessage     | saliene |
|-----------|----------|----------|------------------|--------------------|---------|
| battery   | <        | 10       | BATTERY_CRITICAL | Baterija ispod 10% | 100     |
| battery   | <        | 20       | BATTERY_LOW      | Baterija ispod 20% | 90      |
| battery   | <        | 30       | BATTERY_WARNING  | Baterija ispod 30% | 80      |

| parameter  | operator | threshod | factName       | warningMessage                | saliene |
|------------|----------|----------|----------------|-------------------------------|---------|
| hdop       | >        | 5.0      | GNSS_POOR_HDOP | HDOP > 5, pozicija nepouzdana | 100     |
| satellites | <        | 4        | GNSS_FEW_SATS  | Premalo satelita              | 90      |

Konkretno koje template bi sistem imao:

1. Template za detekciju stanja baterije
2. Template za detekciju stanja GNSS signala
3. Template za detekciju stanja radio signala

Napravio bih jedan genericni template koji bi primao razlicite podatke iz Excel/CSV tabele, tako da mogu da izgenerisem razlicita pravila na osnovu jednog univerzalnog template-a. Ovo je posebno važno u geodetskim misijama gdje različite misije mogu zahtijevati različite pragove. Na primjer, snimanje u planinskom terenu može imati strože GNSS zahtjeve nego ravničarsko snimanje, a operater može da mijenja te pragove bez intervencije programera.

---

## 4. Primjer rezonovanja

1. Ulazni podaci:
  - baterija = 15%
  - signal = slab
  - brzina = visoka
2. Nivo 1:
  - detektuje se nizak nivo baterije
  - detektuje se slab signal
3. Nivo 2:
  - kombinacijom faktora zaključuje se rizično stanje leta
4. Nivo 3:
  - sistem zaključuje da je stanje kritično
  - generiše akciju: RETURN\_HOME
5. CEP:
  - detektuje se više warning događaja u kratkom vremenu
  - dodatno potvrđuje visok rizik
6. Backward chaining:
  - na upit “da li je let bezbjedan?” sistem vraća negativan odgovor